VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3071 - Distributed Systems

ASSIGNMENT REPORT

*dunno what should be for the title

Advisor(s):	Thoại Nam	
Student(s):	Bùi Trần Quyền	ID 2353033
	Nguyễn Đức Phi Long	ID 2452689
	Trần Đức Phú	ID 2452971

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Introduction **5**
1.1 Background and motivation . . . 5
1.2 Problem statement . . . 5
1.3 Objectives . . . 5
1.4 Scope and limitations . . . 5
1.5 Report structure . . . 5

2 Background and Theory **6**
2.1 Definition . . . 6
2.1.1 Basic Crawling Process . . . 6
2.1.2 Main Components . . . 6
2.1.3 Crawling Policies . . . 7
2.1.4 Why Distribute the Crawler? . . . 7
2.2 Concepts and Knowledge from the course . . . 7
2.2.1 Distributed System Model . . . 7
2.2.2 Coordinator–Worker Architecture . . . 8
2.2.3 Distributed Communication . . . 8
2.2.4 Task Allocation and Load Balancing . . . 8
2.2.5 Concurrency and Coordination . . . 8
2.2.6 Fault Tolerance . . . 8
2.2.7 Consistency and the CAP Theorem . . . 9
2.2.8 Scalability Metrics . . . 9
2.2.9 Summary: Concepts and Course Topics . . . 9
2.3 Related work or existing systems . . . 10

3 System Design **11**
3.1 Requirements . . . 11
3.2 Overall architecture . . . 11
3.3 Component design . . . 11
3.4 Communication protocol and API . . . 11
3.5 Key mechanisms . . . 11
3.6 Design decisions and trade-offs . . . 11

4 Implementation **12**
4.1 Technology stack and platform 12
4.2 Project structure 12
4.3 Implementation details of the main components 12
4.4 Challenges 12

5 Experiments and Evaluation **13**
5.1 Experimental setup 13
5.2 Scalability 13
5.3 Static vs. dynamic allocation comparison 13
5.4 Fault tolerance test 13
5.5 Discussion 13

6 Conclusion **14**
6.1 Summary against the objectives 14
6.2 Limitations 14

7 References **15**



1 Introduction

1.1 Background and motivation

1.2 Problem statement

1.3 Objectives

1.4 Scope and limitations

1.5 Report structure

2 Background and Theory

2.1 Definition

A web crawler (also called a spider or bot) is a program that automatically browses the World Wide Web. Starting from a set of **seed URLs**, it downloads web pages, extracts data and hyperlinks from them, and follows those links to discover further pages. Search engines use crawlers to build their indexes, and other common applications include price monitoring, data collection for machine learning, and website archiving.

2.1.1 Basic Crawling Process

A crawler repeats the following loop until its frontier is empty or a stopping condition is reached:

1. Take a URL from the **frontier**, the queue of URLs waiting to be crawled.
2. Download the page over HTTP.
3. Parse the HTML and extract useful content (for example the title and text) and the outgoing links.
4. Normalize the extracted links and discard those that have already been seen.
5. Add the new links to the frontier, store the results, and repeat.

2.1.2 Main Components

- **Frontier:** the queue of URLs to be crawled.
- **Fetcher:** the HTTP client that downloads pages.
- **Parser:** extracts content and links from the HTML.
- **Duplicate detector:** a registry of URLs that were already discovered or processed.
- **Storage:** persists the crawled data and the crawl status.

2.1.3 Crawling Policies

A practical crawler must follow several policies:

- **Politeness:** respect the `robots.txt` file of each site and limit the request rate per domain so that the target server is not overloaded.
- **Depth and URL limits:** bound the crawl to avoid infinite crawling caused by endlessly generated links.
- **URL normalization:** treat different spellings of the same address as a single URL.
- **Robustness:** handle timeouts, redirects, malformed HTML, and unavailable servers.

2.1.4 Why Distribute the Crawler?

A single machine is limited by its network bandwidth, CPU, and memory. In addition, websites respond at very different speeds, so one slow download can delay all following work in a sequential crawler. Distributing the crawl over several workers increases throughput, but it introduces typical distributed-systems problems:

- How are URLs divided among the workers?
- How can the system avoid crawling the same page more than once?
- What happens when a worker, or the coordinator, fails?
- At which point does adding more workers stop improving performance?

These questions are the focus of this project.

2.2 Concepts and Knowledge from the course

This section presents the distributed-systems concepts from the course that are needed to design and evaluate the distributed crawler.

2.2.1 Distributed System Model

A distributed system consists of independent nodes that communicate only by passing messages over a network, with no shared memory and no shared clock. The network may delay or drop messages, and nodes may crash. In this project, the nodes are a coordinator and several crawler workers, and the failure model considered is the crash failure of a worker (and, as an extension, of the coordinator).

2.2.2 Coordinator–Worker Architecture

In a coordinator–worker (master–slave) architecture, one node manages the global state and distributes tasks, while the other nodes execute them. This design is simple and makes global decisions, such as duplicate detection, easy. Its main drawbacks are that the coordinator is a potential **single point of failure** and a possible **performance bottleneck**.

2.2.3 Distributed Communication

The coordinator and the workers communicate with a request–response pattern. In this project, HTTP with the REST style and JSON messages is used: a worker asks the coordinator for a task, downloads the page, and then submits the result and the newly discovered URLs. This corresponds to the client–server model and the remote invocation techniques (RPC and REST APIs) covered in the course.

2.2.4 Task Allocation and Load Balancing

With **static allocation**, each worker receives a fixed number of URLs in advance. Because websites respond at different speeds, some workers finish early and sit idle while others are still busy. With **dynamic allocation**, a worker requests a new task whenever it becomes free, which balances the load automatically. Both strategies are compared in the experiments.

2.2.5 Concurrency and Coordination

If two workers discover the same URL at almost the same time, the coordinator may receive both reports concurrently. Checking whether a URL exists and inserting it into the queue must therefore be a single **atomic** operation; otherwise a **race condition** may put the same URL into the queue twice. In a multithreaded coordinator this requires a mutex or an equivalent synchronization mechanism.

2.2.6 Fault Tolerance

Failures are detected with **heartbeats** and **timeouts**. Each task assigned to a worker has a **lease**: if the worker does not report completion before the lease expires, the coordinator considers the worker failed and returns the task to the queue so that another worker can process it.

Because a worker may finish a task and crash before its report arrives, the same task can be executed twice. The system therefore provides **at-least-once** execution combined with **idempotent** result submission, which means that processing the same result twice does not create duplicate records.

2.2.7 Consistency and the CAP Theorem

The CAP theorem states that during a network partition a distributed system must choose between consistency and availability. A central URL registry favors **consistency**: a worker that cannot reach the coordinator cannot obtain new tasks, so availability is reduced during a partition. This trade-off is discussed in the design chapter.

2.2.8 Scalability Metrics

The scalability of the system is evaluated with the following metrics:

- total crawling time and throughput (pages per second);
- average response time per page;
- coordinator CPU usage and worker utilization.

With T_1 the running time on one worker and T_p the running time on p workers, the speedup and the efficiency are

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p}.$$

Amdahl's law bounds the achievable speedup when a fraction f of the work is sequential (here, mainly the coordinator):

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}.$$

2.2.9 Summary: Concepts and Course Topics

Table **??** relates the problems of the crawler to the concepts of the course.

Problem in the crawler	**Concept**	**Course topic**
Coordinator and workers exchange messages	Client–server, REST, RPC, message passing	Communication
Workers request tasks when free	Dynamic allocation, load balancing	Distributed computing
Two workers report the same URL	Race condition, atomic operation, mutual exclusion	Transactions, coordination
A result is submitted twice	Idempotency, at-least-once execution	Transactions
A worker crashes during a task	Heartbeat, timeout, lease, recovery	Replication, fault tolerance
The coordinator crashes	Persistent state, replication, leader election	Co-ordination, consensus
Shared visited-URL set	Consistency, CAP theorem	Introduction

Table 2.1: Crawler problems and the related distributed-systems concepts.

2.3 Related work or existing systems



3 System Design

3.1 Requirements

3.2 Overall architecture

3.3 Component design

3.4 Communication protocol and API

3.5 Key mechanisms

3.6 Design decisions and trade-offs

4 Implementation

4.1 Technology stack and platform

4.2 Project structure

4.3 Implementation details of the main components

4.4 Challenges



5 Experiments and Evaluation

5.1 Experimental setup

5.2 Scalability

5.3 Static vs. dynamic allocation comparison

5.4 Fault tolerance test

5.5 Discussion

6 Conclusion

6.1 Summary against the objectives

6.2 Limitations



7 References